DATA STRUCTURES & ALGORITHMS

Combined Assignment Code Book

Organized full source listings, topic-wise excerpts, and assignment notes

Included topics	Matrix operations • Sparse matrices • Sorting/searching • Linked lists • Queues
Additional code	Printer Queue Manager using Circular Queue (code shared in chat)
Expression conversion	Problem statement included; implementation source was not supplied yet

Source code is preserved as supplied. The circular-queue program shared in chat is added as a separate full listing. No expression-conversion implementation has been invented or substituted.

Contents

• 1. Topic-wise quick reference

• 2. Assignment 1 — Matrix Operations

• 3. Assignment 2 — Sparse Matrix and Fast Transpose

• 4. Assignment 3 — Student Database: Sorting and Searching

• 5. Assignment 3 — Sample Output

• 6. Assignment 5 — Polynomial Addition Using Circular Linked List

• 7. Assignment 8 — Printer Queue Manager (uploaded source)

• 8. Additional Code — Printer Queue Manager using Circular Queue

• 9. Expression Conversion Using Stack ADT — problem statement

Topic-wise quick reference

Matrix operations

The matrix assignment source is the reference for the operations implemented. The excerpt below highlights the program's matrix-related declarations and logic.

```
void transpose(int a[10][10], int r, int c);

int main()
{
    int a[10][10], b[10][10];
    int r1, c1, r2, c2;
    int choice;

    printf("Enter rows and columns of Matrix 1: ");
    scanf("%d%d", &r1, &c1);

    printf("Enter elements of Matrix 1:\n");
    accept(a, r1, c1);

    printf("Enter rows and columns of Matrix 2: ");
```

Sparse matrix / fast transpose

Sparse matrices store non-zero values with their row and column positions. The supplied Assignment 2 source contains the implementation; the excerpt below is taken directly from it.

```
    }

    printf("Transposed Sparse matrix (Fast Transpose) is: \n");
    for (i = 0; i <= num_terms; i++)
    {
        printf("%d %d %d\n", t[0][i], t[1][i], t[2][i]);
    }

    return 0;
}
```

Student database: sorting and searching

```
            case 2:
                insertionSort(students, n);
                printf("\nStudents sorted using Insertion Sort:\n");
                displayStudents(students, n);
                break;

            case 3:
                selectionSort(students, n);
                printf("\nStudents sorted using Selection Sort:\n");
```

```
                displayStudents(students, n);
                break;

            case 4:
                shellSort(students, n);
                printf("\nStudents sorted using Shell Sort:\n");
                displayStudents(students, n);
                break;
```

Circular linked list: polynomial addition

```
#include <stdlib.h>

struct Node {
    int coeff;
    int exp;
    struct Node *next;
};

typedef struct Node Node;

Node *createNode(int coeff, int exp) {
    Node *newNode = (Node *)malloc(sizeof(Node));

    if (newNode == NULL) {
        printf("Memory allocation failed!\n");
        exit(1);
    }
```

Queue operations

The shared printer-manager code uses a circular queue: enqueue adds at rear, dequeue processes the front job, and display traverses from front to rear with wrap-around.

```
if ((rear + 1) % MAX == front)
{
    printf("\nQueue Overflow! Print queue is full.\n");
    return;
}
```

Assignment 1 — Matrix Operations

Original file: 1262251789_assi1 code.c

```c
#include <stdio.h>

void accept(int a[10][10], int r, int c);
void display(int a[10][10], int r, int c);
void addition(int a[10][10], int b[10][10], int r, int c);
void subtraction(int a[10][10], int b[10][10], int r, int c);
void multiplication(int a[10][10], int b[10][10], int r1, int c1, int c2);
void transpose(int a[10][10], int r, int c);

int main()
{
    int a[10][10], b[10][10];
    int r1, c1, r2, c2;
    int choice;

    printf("Enter rows and columns of Matrix 1: ");
    scanf("%d%d", &r1, &c1);

    printf("Enter elements of Matrix 1:\n");
    accept(a, r1, c1);

    printf("Enter rows and columns of Matrix 2: ");
    scanf("%d%d", &r2, &c2);

    printf("Enter elements of Matrix 2:\n");
    accept(b, r2, c2);

    do
    {
        printf("\n----- MENU -----\n");
        printf("1. Addition\n");
        printf("2. Subtraction\n");
        printf("3. Multiplication\n");
        printf("4. Transpose of Matrix 1\n");
        printf("5. Display Matrices\n");
        printf("6. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch(choice)
        {
            case 1:
                if(r1 == r2 && c1 == c2)
                    addition(a, b, r1, c1);
                else
                    printf("Addition not possible.\n");
                break;

            case 2:
                if(r1 == r2 && c1 == c2)
                    subtraction(a, b, r1, c1);
                else
                    printf("Subtraction not possible.\n");
                break;

            case 3:
                if(c1 == r2)
                    multiplication(a, b, r1, c1, c2);
                else
                    printf("Multiplication not possible.\n");
                break;

            case 4:
                transpose(a, r1, c1);
                break;

            case 5:
                printf("\nMatrix 1:\n");
                display(a, r1, c1);

                printf("\nMatrix 2:\n");
                display(b, r2, c2);
                break;

            case 6:
                printf("Exiting...\n");
                break;

            default:
```

```
                printf("Invalid Choice!\n");
        }

    } while(choice != 6);

    return 0;
}

void accept(int a[10][10], int r, int c)
{
    int i, j;

    for(i = 0; i < r; i++)
    {
        for(j = 0; j < c; j++)
        {
            scanf("%d", &a[i][j]);
        }
    }
}

void display(int a[10][10], int r, int c)
{
    int i, j;

    for(i = 0; i < r; i++)
    {
        for(j = 0; j < c; j++)
        {
            printf("%4d", a[i][j]);
        }
        printf("\n");
    }
}

void addition(int a[10][10], int b[10][10], int r, int c)
{
    int i, j, sum[10][10];

    printf("\nAddition:\n");

    for(i = 0; i < r; i++)
    {
        for(j = 0; j < c; j++)
        {
            sum[i][j] = a[i][j] + b[i][j];
            printf("%4d", sum[i][j]);
        }
        printf("\n");
    }
}

void subtraction(int a[10][10], int b[10][10], int r, int c)
{
    int i, j, sub[10][10];

    printf("\nSubtraction:\n");

    for(i = 0; i < r; i++)
    {
        for(j = 0; j < c; j++)
        {
            sub[i][j] = a[i][j] - b[i][j];
            printf("%4d", sub[i][j]);
        }
        printf("\n");
    }
}

void multiplication(int a[10][10], int b[10][10], int r1, int c1, int c2)
{
    int i, j, k;
    int mul[10][10];

    for(i = 0; i < r1; i++)
    {
        for(j = 0; j < c2; j++)
        {
            mul[i][j] = 0;

            for(k = 0; k < c1; k++)
            {
                mul[i][j] += a[i][k] * b[k][j];
            }
        }
```

```
    }

    printf("\nMultiplication:\n");

    for(i = 0; i < r1; i++)
    {
        for(j = 0; j < c2; j++)
        {
            printf("%4d", mul[i][j]);
        }
        printf("\n");
    }
}

void transpose(int a[10][10], int r, int c)
{
    int i, j;

    printf("\nTranspose of Matrix 1:\n");

    for(i = 0; i < c; i++)
    {
        for(j = 0; j < r; j++)
        {
            printf("%4d", a[j][i]);
        }
        printf("\n");
    }
}
```

Assignment 2 — Sparse Matrix and Fast Transpose

Original file: 1262251789_assi 2 code.c

```c
#include <stdio.h>

int main()
{
    int a[10][10], s[3][101], t[3][101];
    int m, n, i, j, k = 1;

    printf("Enter rows and columns of matrix: ");
    scanf("%d %d", &m, &n);

    if (m > 10 || n > 10 || m <= 0 || n <= 0) {
        printf("Error: Matrix dimensions must be between 1 and 10.\n");
        return 1;
    }

    printf("Enter row and columns value of the matrix: \n");
    for (i = 0; i < m; i++)
    {
        for (j = 0; j < n; j++)
        {
            scanf("%d", &a[i][j]);
            if (a[i][j] != 0)
            {
                s[0][k] = i;
                s[1][k] = j;
                s[2][k] = a[i][j];
                k++;
            }
        }
    }

    s[0][0] = m;
    s[1][0] = n;
    s[2][0] = k - 1;

    printf("Sparse matrix is: \n");
    for (i = 0; i < k; i++)
    {
        printf("%d %d %d\n", s[0][i], s[1][i], s[2][i]);
    }


    t[0][0] = s[1][0];
    t[1][0] = s[0][0];
    t[2][0] = s[2][0];
    int num_cols = s[1][0];
    int num_terms = s[2][0];

    if (num_terms > 0)
    {
        int count[100] = {0};
        int start_pos[100] = {0};

        for (i = 1; i <= num_terms; i++)
        {
            count[s[1][i]]++;
        }


        start_pos[0] = 1;
        for (i = 1; i < num_cols; i++)
        {
            start_pos[i] = start_pos[i - 1] + count[i - 1];
        }


        for (i = 1; i <= num_terms; i++)
        {
            int col = s[1][i];
            int pos = start_pos[col];

            t[0][pos] = s[1][i];
            t[1][pos] = s[0][i];
            t[2][pos] = s[2][i];
            start_pos[col]++;
        }
    }

    printf("Transposed Sparse matrix (Fast Transpose) is: \n");
```

```
    for (i = 0; i <= num_terms; i++)
    {
        printf("%d %d %d\n", t[0][i], t[1][i], t[2][i]);
    }

    return 0;
}
```

Assignment 3 — Student Database: Sorting and Searching

Original file: 1262251789_assi 3 code.c

```c
#include <stdio.h>
#include <string.h>

#define MAX 100

// Structure for student
struct Student
{
    int rollNumber;
    char name[50];
    int marks[3];
    int totalMarks;
    float percentage;
};

// Function declarations
void acceptStudents(struct Student students[], int n);
void displayStudents(struct Student students[], int n);

void insertionSort(struct Student students[], int n);
void selectionSort(struct Student students[], int n);
void shellSort(struct Student students[], int n);

int linearSearch(struct Student students[], int n, int key);
int binarySearch(struct Student students[], int n, int key);


// Main function
int main()
{
    struct Student students[MAX];
    int n, choice, key, result;

    printf("Enter number of students: ");
    scanf("%d", &n);

    acceptStudents(students, n);

    do
    {
        printf("\n========== STUDENT DATABASE ==========\n");
        printf("1. Display Students\n");
        printf("2. Insertion Sort\n");
        printf("3. Selection Sort\n");
        printf("4. Shell Sort\n");
        printf("5. Linear Search\n");
        printf("6. Binary Search\n");
        printf("7. Exit\n");

        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch(choice)
        {
            case 1:
                displayStudents(students, n);
                break;

            case 2:
                insertionSort(students, n);
                printf("\nStudents sorted using Insertion Sort:\n");
                displayStudents(students, n);
                break;

            case 3:
                selectionSort(students, n);
                printf("\nStudents sorted using Selection Sort:\n");
                displayStudents(students, n);
                break;

            case 4:
                shellSort(students, n);
                printf("\nStudents sorted using Shell Sort:\n");
                displayStudents(students, n);
                break;

            case 5:
                printf("\nEnter Roll Number to search: ");
                scanf("%d", &key);
```

```
                result = linearSearch(students, n, key);

                if(result != -1)
                {
                    printf("\nStudent Found!\n");
                    printf("Roll Number: %d\n", students[result].rollNumber);
                    printf("Name: %s\n", students[result].name);
                    printf("Total Marks: %d\n", students[result].totalMarks);
                    printf("Percentage: %.2f%%\n",
                           students[result].percentage);
                }
                else
                {
                    printf("\nStudent not found.\n");
                }
                break;

            case 6:
                // Binary search requires sorted data
                insertionSort(students, n);

                printf("\nStudents sorted by Roll Number:\n");
                displayStudents(students, n);

                printf("\nEnter Roll Number to search: ");
                scanf("%d", &key);

                result = binarySearch(students, n, key);

                if(result != -1)
                {
                    printf("\nStudent Found!\n");
                    printf("Roll Number: %d\n", students[result].rollNumber);
                    printf("Name: %s\n", students[result].name);
                    printf("Total Marks: %d\n", students[result].totalMarks);
                    printf("Percentage: %.2f%%\n",
                           students[result].percentage);
                }
                else
                {
                    printf("\nStudent not found.\n");
                }
                break;

            case 7:
                printf("\nProgram terminated.\n");
                break;

            default:
                printf("\nInvalid choice!\n");
        }

    } while(choice != 7);

    return 0;
}


// Accept student details
void acceptStudents(struct Student students[], int n)
{
    int i, j, sum;

    for(i = 0; i < n; i++)
    {
        printf("\nEnter details of Student %d\n", i + 1);

        printf("Enter Roll Number: ");
        scanf("%d", &students[i].rollNumber);

        printf("Enter Name: ");
        scanf(" %[^\n]", students[i].name);

        printf("Enter marks in 3 subjects: ");

        sum = 0;

        for(j = 0; j < 3; j++)
        {
            scanf("%d", &students[i].marks[j]);
            sum = sum + students[i].marks[j];
        }

        students[i].totalMarks = sum;
```

```
        students[i].percentage =
            (sum / 300.0) * 100;
    }
}


// Display students
void displayStudents(struct Student students[], int n)
{
    int i;

    printf("\n-------------------------------------------------\n");
    printf("Roll No\tName\t\tTotal\tPercentage\n");
    printf("-------------------------------------------------\n");

    for(i = 0; i < n; i++)
    {
        printf("%d\t%-15s\t%d\t%.2f%%\n",
               students[i].rollNumber,
               students[i].name,
               students[i].totalMarks,
               students[i].percentage);
    }

    printf("-------------------------------------------------\n");
}


// Insertion Sort
void insertionSort(struct Student students[], int n)
{
    int i, j;
    struct Student key;

    for(i = 1; i < n; i++)
    {
        key = students[i];
        j = i - 1;

        while(j >= 0 &&
              students[j].rollNumber > key.rollNumber)
        {
            students[j + 1] = students[j];
            j--;
        }

        students[j + 1] = key;
    }
}


// Selection Sort
void selectionSort(struct Student students[], int n)
{
    int i, j, min;
    struct Student temp;

    for(i = 0; i < n - 1; i++)
    {
        min = i;

        for(j = i + 1; j < n; j++)
        {
            if(students[j].rollNumber <
               students[min].rollNumber)
            {
                min = j;
            }
        }

        if(min != i)
        {
            temp = students[i];
            students[i] = students[min];
            students[min] = temp;
        }
    }
}


// Shell Sort
void shellSort(struct Student students[], int n)
{
    int gap, i, j;
    struct Student temp;
```

```
    for(gap = n / 2; gap > 0; gap = gap / 2)
    {
        for(i = gap; i < n; i++)
        {
            temp = students[i];

            for(j = i;
                j >= gap &&
                students[j - gap].rollNumber > temp.rollNumber;
                j = j - gap)
            {
                students[j] = students[j - gap];
            }

            students[j] = temp;
        }
    }
}


// Linear Search
int linearSearch(struct Student students[], int n, int key)
{
    int i;

    for(i = 0; i < n; i++)
    {
        if(students[i].rollNumber == key)
        {
            return i;
        }
    }

    return -1;
}


// Binary Search
int binarySearch(struct Student students[], int n, int key)
{
    int low = 0;
    int high = n - 1;
    int mid;

    while(low <= high)
    {
        mid = (low + high) / 2;

        if(students[mid].rollNumber == key)
        {
            return mid;
        }
        else if(key < students[mid].rollNumber)
        {
            high = mid - 1;
        }
        else
        {
            low = mid + 1;
        }
    }

    return -1;
}
```

Assignment 3 — Sample Output

Original file: 1262251789_assi3 output.txt

Output text as supplied:

```
Enter number of students: 3

Enter details of Student 1
Enter Roll Number: 103
Enter Name: Rahul
Enter marks in 3 subjects: 78 85 90

Enter details of Student 2
Enter Roll Number: 101
Enter Name: Priya
Enter marks in 3 subjects: 88 92 95

Enter details of Student 3
Enter Roll Number: 102
Enter Name: Amit
Enter marks in 3 subjects: 75 80 85

========== STUDENT DATABASE ==========
1. Display Students
2. Insertion Sort
3. Selection Sort
4. Shell Sort
5. Linear Search
6. Binary Search
7. Exit
Enter your choice: 1

-------------------------------------------------
Roll No Name            Total   Percentage
-------------------------------------------------
103     Rahul           253     84.33%
101     Priya           275     91.67%
102     Amit            240     80.00%
-------------------------------------------------

========== STUDENT DATABASE ==========
1. Display Students
2. Insertion Sort
3. Selection Sort
4. Shell Sort
5. Linear Search
6. Binary Search
7. Exit
Enter your choice: 2

Students sorted using Insertion Sort:

-------------------------------------------------
Roll No Name            Total   Percentage
-------------------------------------------------
101     Priya           275     91.67%
102     Amit            240     80.00%
103     Rahul           253     84.33%
-------------------------------------------------

========== STUDENT DATABASE ==========
1. Display Students
2. Insertion Sort
3. Selection Sort
4. Shell Sort
5. Linear Search
6. Binary Search
7. Exit
Enter your choice: 3

Students sorted using Selection Sort:

-------------------------------------------------
Roll No Name            Total   Percentage
-------------------------------------------------
101     Priya           275     91.67%
102     Amit            240     80.00%
103     Rahul           253     84.33%
-------------------------------------------------

========== STUDENT DATABASE ==========
1. Display Students
```

```
2. Insertion Sort
3. Selection Sort
4. Shell Sort
5. Linear Search
6. Binary Search
7. Exit
Enter your choice: 4

Students sorted using Shell Sort:

-------------------------------------------------
Roll No Name            Total   Percentage
-------------------------------------------------
101     Priya           275     91.67%
102     Amit            240     80.00%
103     Rahul           253     84.33%
-------------------------------------------------

========== STUDENT DATABASE ==========
1. Display Students
2. Insertion Sort
3. Selection Sort
4. Shell Sort
5. Linear Search
6. Binary Search
7. Exit
Enter your choice: 5

Enter Roll Number to search: 102

Student Found!
Roll Number: 102
Name: Amit
Total Marks: 240
Percentage: 80.00%

========== STUDENT DATABASE ==========
1. Display Students
2. Insertion Sort
3. Selection Sort
4. Shell Sort
5. Linear Search
6. Binary Search
7. Exit
Enter your choice: 6

Students sorted by Roll Number:

-------------------------------------------------
Roll No Name            Total   Percentage
-------------------------------------------------
101     Priya           275     91.67%
102     Amit            240     80.00%
103     Rahul           253     84.33%
-------------------------------------------------

Enter Roll Number to search: 101

Student Found!
Roll Number: 101
Name: Priya
Total Marks: 275
Percentage: 91.67%

========== STUDENT DATABASE ==========
1. Display Students
2. Insertion Sort
3. Selection Sort
4. Shell Sort
5. Linear Search
6. Binary Search
7. Exit
Enter your choice: 7

Program terminated.
```

Assignment 5 — Polynomial Addition Using Circular Linked List

Original file: 1262251789_assi5 code.txt

```
Assignment 5 - Circular linked list

#include <stdio.h>
#include <stdlib.h>

struct Node {
    int coeff;
    int exp;
    struct Node *next;
};

typedef struct Node Node;

Node *createNode(int coeff, int exp) {
    Node *newNode = (Node *)malloc(sizeof(Node));

    if (newNode == NULL) {
        printf("Memory allocation failed!\n");
        exit(1);
    }

    newNode->coeff = coeff;
    newNode->exp = exp;
    newNode->next = newNode;

    return newNode;
}

// Insert a term at the end of the circular linked list
Node *insertTerm(Node *head, int coeff, int exp) {
    if (coeff == 0)
        return head;

    Node *newNode = createNode(coeff, exp);

    if (head == NULL) {
        return newNode;
    }

    Node *temp = head;

    while (temp->next != head) {
        temp = temp->next;
    }

    temp->next = newNode;
    newNode->next = head;

    return head;
}

// Create a polynomial
Node *createPolynomial() {
    Node *head = NULL;
    int n, coeff, exp;

    printf("Enter number of terms: ");
    scanf("%d", &n);

    for (int i = 0; i < n; i++) {
        printf("Enter coefficient and exponent: ");
        scanf("%d %d", &coeff, &exp);

        head = insertTerm(head, coeff, exp);
    }

    return head;
}

// Display polynomial
void displayPolynomial(Node *head) {
    if (head == NULL) {
        printf("0\n");
        return;
    }
```

```
    Node *temp = head;
    int first = 1;

    do {
        int c = temp->coeff;
        int e = temp->exp;

        if (c != 0) {
            if (!first) {
                if (c > 0)
                    printf(" + ");
                else
                    printf(" - ");
            } else if (c < 0) {
                printf("-");
            }

            int absCoeff = abs(c);

            if (e == 0) {
                printf("%d", absCoeff);
            } else {
                if (absCoeff != 1)
                    printf("%d", absCoeff);

                printf("x");

                if (e != 1)
                    printf("^%d", e);
            }

            first = 0;
        }

        temp = temp->next;
    } while (temp != head);

    if (first)
        printf("0");

    printf("\n");
}

// Add two polynomials
Node *addPolynomial(Node *p1, Node *p2) {
    Node *result = NULL;

    if (p1 != NULL) {
        Node *temp = p1;

        do {
            result = insertTerm(result,
                                temp->coeff, temp->exp);
            temp = temp->next;
        } while (temp != p1);
    }

    if (p2 != NULL) {
        Node *temp = p2;

        do {
            Node *current = result;
            Node *match = NULL;

            if (current != NULL) {
                do {
                    if (current->exp == temp->exp) {
                        match = current;
                        break;
                    }
                    current = current->next;
                } while (current != result);
            }

            if (match != NULL) {
                match->coeff += temp->coeff;
            } else {
                result = insertTerm(result,
                                    temp->coeff, temp->exp);
            }

            temp = temp->next;
        } while (temp != p2);
    }
```

```
    // Remove terms whose coefficients become zero
    if (result != NULL) {
        Node *prev = result;
        while (prev->next != result)
            prev = prev->next;

        Node *current = result;

        do {
            Node *nextNode = current->next;

            if (current->coeff == 0) {
                if (current->next == current) {
                    free(current);
                    return NULL;
                }

                prev->next = nextNode;

                if (current == result)
                    result = nextNode;

                free(current);
            } else {
                prev = current;
            }

            current = nextNode;
        } while (current != result);
    }

    return result;
}

int main() {
    Node *poly1 = NULL;
    Node *poly2 = NULL;
    Node *sum = NULL;

    printf("Create First Polynomial:\n");
    poly1 = createPolynomial();

    printf("\nCreate Second Polynomial:\n");
    poly2 = createPolynomial();

    printf("\nFirst Polynomial: ");
    displayPolynomial(poly1);

    printf("Second Polynomial: ");
    displayPolynomial(poly2);

    sum = addPolynomial(poly1, poly2);

    printf("Resultant Polynomial: ");
    displayPolynomial(sum);

    return 0;
}
```

Assignment 8 — Printer Queue Manager (provided source)

Original file: 1262251789_assi8 code.c

```
#include <stdio.h>
#include <string.h>

#define MAX 5

struct PrintJob
{
    int jobID;
    char title[50];
};

struct PrintJob queue[MAX];

int front = -1;
int rear = -1;

// Enqueue
void enqueue()
{
    struct PrintJob job;

    if (rear == MAX - 1)
    {
        printf("\nQueue Overflow! Print queue is full.\n");
        return;
    }

    printf("\nEnter Job ID: ");
    scanf("%d", &job.jobID);

    printf("Enter Document Title: ");
    scanf("%s", job.title);

    if (front == -1)
        front = 0;

    rear++;

    queue[rear] = job;

    printf("Print job added successfully.\n");
}

// Dequeue
void dequeue()
{
    if (front == -1 || front > rear)
    {
        printf("\nQueue Underflow! No print jobs available.\n");
        return;
    }

    printf("\nProcessed Print Job:");
    printf("\nJob ID: %d", queue[front].jobID);
    printf("\nDocument Title: %s\n", queue[front].title);

    front++;

    if (front > rear)
    {
        front = -1;
        rear = -1;
    }
}

// Display
void display()
{
    int i;

    if (front == -1)
    {
        printf("\nNo Pending Print Jobs.\n");
        return;
    }

    printf("\nPending Print Jobs:\n");

    for (i = front; i <= rear; i++)
```

```
    {
        printf("\nJob ID: %d", queue[i].jobID);
        printf("\nDocument Title: %s\n", queue[i].title);
    }
}

// Main
int main()
{
    int choice;

    printf("===== PRINTER QUEUE MANAGER - LINEAR QUEUE =====\n");

    while (1)
    {
        printf("\n\n1. Enqueue Print Job");
        printf("\n2. Dequeue Print Job");
        printf("\n3. Display Print Jobs");
        printf("\n4. Exit");

        printf("\n\nEnter your choice: ");
        scanf("%d", &choice);

        switch (choice)
        {
            case 1:
                enqueue();
                break;

            case 2:
                dequeue();
                break;

            case 3:
                display();
                break;

            case 4:
                printf("\nProgram terminated successfully.\n");
                return 0;

            default:
                printf("\nInvalid choice!\n");
        }
    }

    return 0;
}
```

Additional Code — Printer Queue Manager using Circular Queue

Source: code pasted in chat. This is a circular queue implementation with capacity MAX = 5 and menu options to enqueue, dequeue, display, and exit.

```
#include <stdio.h>
#include <string.h>

#define MAX 5

struct PrintJob
{
    int jobID;
    char title[50];
};

struct PrintJob queue[MAX];

int front = -1;
int rear = -1;

// Enqueue
void enqueue()
{
    struct PrintJob job;

    // Check overflow
    if ((rear + 1) % MAX == front)
    {
        printf("\nQueue Overflow! Print queue is full.\n");
        return;
    }

    printf("\nEnter Job ID: ");
    scanf("%d", &job.jobID);

    printf("Enter Document Title: ");
    scanf("%s", job.title);

    // First element
    if (front == -1)
    {
        front = 0;
        rear = 0;
    }
    else
    {
        rear = (rear + 1) % MAX;
    }

    queue[rear] = job;

    printf("Print job added successfully.\n");
}

// Dequeue
void dequeue()
{
    if (front == -1)
    {
        printf("\nQueue Underflow! No print jobs available.\n");
        return;
    }

    printf("\nProcessed Print Job:");
    printf("\nJob ID: %d", queue[front].jobID);
    printf("\nDocument Title: %s\n", queue[front].title);

    // Only one element
    if (front == rear)
    {
        front = -1;
        rear = -1;
    }
    else
    {
        front = (front + 1) % MAX;
    }
}
```

```
// Display
void display()
{
    int i;

    if (front == -1)
    {
        printf("\nNo Pending Print Jobs.\n");
        return;
    }

    printf("\nPending Print Jobs:\n");

    i = front;

    while (1)
    {
        printf("\nJob ID: %d", queue[i].jobID);
        printf("\nDocument Title: %s\n", queue[i].title);

        if (i == rear)
            break;

        i = (i + 1) % MAX;
    }
}

// Main
int main()
{
    int choice;

    printf("===== PRINTER QUEUE MANAGER - CIRCULAR QUEUE =====\n");

    while (1)
    {
        printf("\n\n1. Enqueue Print Job");
        printf("\n2. Dequeue Print Job");
        printf("\n3. Display Print Jobs");
        printf("\n4. Exit");

        printf("\n\nEnter your choice: ");
        scanf("%d", &choice);

        switch (choice)
        {
            case 1:
                enqueue();
                break;

            case 2:
                dequeue();
                break;

            case 3:
                display();
                break;

            case 4:
                printf("\nProgram terminated successfully.\n");
                return 0;

            default:
                printf("\nInvalid choice!\n");
        }
    }

    return 0;
}
```

Expression Conversion Using Stack ADT

Problem statement

Implement stack as an ADT and apply it for different expression conversions.

Required conversion choices

• Infix to Postfix

• Infix to Prefix

• Prefix to Postfix

• Prefix to Infix

• Postfix to Infix

• Postfix to Prefix

Note: The expression-conversion assignment's C implementation was not included in the files or code pasted so far. This section records the problem statement only; it does not fabricate a program.

Suggested topic checklist for the implementation

• Stack ADT operations: push, pop, peek/top, isEmpty, and overflow/underflow checks.

• Operator precedence and associativity handling.

• Expression scanning in the correct direction for each conversion.

• Parentheses and multi-character operands, if required by the course specification.

• Sample input/output for each supported conversion.